

COURSE NAME: Database Management Systems COURSE CODE: CSA05 COURSE OUTCOME

COVERED CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6) Activity Tool : Analytical

Problem Solving (High) Assessment Tool Weightage: 35%

NAME:K.VIJAYALAKSHMI

REG.NO:192511189

Question 1

Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

EmpName
Arun
Priya
Rahul
Sneha
Vijay

5 rows in set (0.00 sec)

```
mysql> SELECT *  
-> FROM Employee  
-> WHERE Salary > 50000;
```

EmpID	EmpName	Salary	DeptID
101	Arun	55000.00	D1
103	Rahul	65000.00	D1
105	Vijay	70000.00	D2

3 rows in set (0.00 sec)

EmpName	DeptName
Arun	IT
Rahul	IT
Priya	CSE
Vijay	CSE
Sneha	ECE

5 rows in set (0.00 sec)

```
=> CROSS JOIN Department;
```

EmpID	EmpName	Salary	DeptID	DeptID	DeptName
101	Arun	55000.00	D1	D3	ECE
101	Arun	55000.00	D1	D2	CSE
101	Arun	55000.00	D1	D1	IT
102	Priya	45000.00	D2	D3	ECE
102	Priya	45000.00	D2	D2	CSE
102	Priya	45000.00	D2	D1	IT
103	Rahul	65000.00	D1	D3	ECE
103	Rahul	65000.00	D1	D2	CSE
103	Rahul	65000.00	D1	D1	IT
104	Sneha	40000.00	D3	D3	ECE
104	Sneha	40000.00	D3	D2	CSE
104	Sneha	40000.00	D3	D1	IT
105	Vijay	70000.00	D2	D3	ECE
105	Vijay	70000.00	D2	D2	CSE
105	Vijay	70000.00	D2	D1	IT

15 rows in set (0.00 sec)

EmpID	EmpName	Salary	DeptID
102	Priya	45000.00	D2
105	Vijay	70000.00	D2
104	Sneha	40000.00	D3

3 rows in set (0.00 sec)

Question 2

Write SQL DDL statements to create STUDENT and DEPARTMENT tables with appropriate constraints.

```
mysql> SELECT * FROM Student;
```

StudentID	Name	DOB	DeptID
101	Arun	2005-04-15	D1
102	Priya	2004-09-20	D2
103	Rahul	2005-01-10	D1
104	Sneha	2004-12-05	D3

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| D1     | IT       |
| D2     | CSE      |
| D3     | ECE      |
+-----+-----+
3 rows in set (0.00 sec)
```

Question 3

Formulate SQL queries using GROUP BY, HAVING, COUNT, SUM, AVG, MAX and MIN.

```
mysql> SELECT Category,
-> COUNT(*) AS TotalProducts
-> FROM Sales
-> GROUP BY Category;
+-----+-----+
| Category | TotalProducts |
+-----+-----+
| Electronics | 3 |
| Furniture | 2 |
| Stationery | 2 |
+-----+-----+
3 rows in set (0.00 sec)
```

```
-> GROUP BY Category;
+-----+-----+
| Category | TotalSales |
+-----+-----+
| Electronics | 56700.00 |
| Furniture | 9500.00 |
| Stationery | 150.00 |
+-----+-----+
```

Category	AverageSales
Electronics	18900.000000
Furniture	4750.000000
Stationery	75.000000

Category	HighestSale
Electronics	55000.00
Furniture	6000.00
Stationery	100.00

3 rows in set (0.00 sec)

```
mysql> SELECT Category,
-> MIN(Amount) AS LowestSale
-> FROM Sales
-> GROUP BY Category;
```

Category	LowestSale
Electronics	500.00
Furniture	3500.00
Stationery	50.00

Question 4

Write a correlated subquery to find employees earning more than the average salary of their own department.

```
-> );
```

EmpID	EmpName	DeptID	Salary
101	Arun	D1	60000.00
103	Rahul	D2	70000.00
105	Vijay	D3	55000.00

3 rows in set (0.00 sec)

Question 5

Apply all JOIN operations on Employee and Project tables.

```
+-----+
| EmpID | EmpName |
+-----+
| 101   | Arun    |
| 102   | Priya   |
| 103   | Rahul   |
| 105   | Vijay   |
+-----+
4 rows in set (0.00 sec)
```

```
-> );
+-----+
| EmpID | EmpName |
+-----+
| 104   | Sneha   |
| 106   | Kavın   |
+-----+
2 rows in set (0.00 sec)
```

6.Views

- **CREATE VIEW**
- **SELECT * FROM ViewName**

```
mysql> SELECT * FROM IT_Employees;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | D1     | 60000.00 |
| 102   | Priya   | D1     | 45000.00 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

7. Stored Procedure

- **CREATE PROCEDURE**
- **CALL ProcedureName()**

```
mysql> CALL GetEmployeesByDepartment('D1');
```

EmpID	EmpName	DeptID	Salary
101	Arun	D1	60000.00
102	Priya	D1	45000.00

```
2 rows in set (0.00 sec)
```

8. Trigger

- CREATE TRIGGER
- AFTER INSERT
- UPDATE Stock

```
mysql> SELECT * FROM ProductStock;
```

ProductID	ProductName	Stock
P1	Laptop	15
P2	Mouse	50
P3	Keyboard	40

9. . Function

- CREATE FUNCTION
- SELECT FunctionName()

EmpID	EmpName	MonthlySalary	AnnualSalary
101	Arun	60000.00	720000.00
102	Priya	45000.00	540000.00
103	Rahul	70000.00	840000.00
104	Sneha	50000.00	600000.00
105	Vijay	55000.00	660000.00
106	Kavin	40000.00	480000.00

10. Transaction Control

- **START TRANSACTION**
- **UPDATE**
- **ROLLBACK**
- **COMMIT**

```
mysql> SELECT * FROM BankAccount;
+-----+-----+-----+
| AccountNo | CustomerName | Balance |
+-----+-----+-----+
| A101      | Arun         | 45000.00 |
| A102      | Priya        | 40000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Code of Conduct:

I _____ **K.VIJAYALAKSHMI(192511189)** _____ (Name / Reg No) certify that this submission is my

original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**K.VIJAYALAKSHMI**